

Lab-6 Ex5 - Finding π Using the Nilakantha's Series

Nilakantha series is a way to compute π using an infinite series. This amazing formula was documented by Indian polymath Nilakantha Somayaji around 1500AD.

Write a C program to ask the user for an input integer n which must be within 1 and 100 inclusively. No input validation on n is required. Evaluate the Nilakantha series up to n terms, but only print out the sums **up to the 1st term, up to the 2nd term, up to the second last term, and up to the last term** (i.e., maximum 4 sums). Please see the sample runs for reference and explanation.

Nilakantha series is given as:

$$\pi = 3 + 4/(2*3*4) - 4/(4*5*6) + 4/(6*7*8) - 4/(8*9*10) + 4/(10*11*12) - 4/(12*13*14) \dots$$

Note that the sign of the terms are alternating. If we list out term by term, $T_1, T_2, \dots, T_n$ it will be like this:

$$\text{1st Term, } T_1 = 3$$

$$\text{2nd Term, } T_2 = +4/(2*3*4)$$

$$\text{3rd Term, } T_3 = -4/(4*5*6)$$

$$\text{4th Term, } T_4 = +4/(6*7*8)$$

...

Look at the formula for T_2 to T_4 . Can you see the pattern? To successfully write your C program, you have to find out the general formula for T_i , where i is an integer greater than 1.

$$i\text{'th Term, } T_i = \pm ??? / (?? * ?? * ??)$$

Please derive the answers to the ?? above on paper BEFORE you write the program.

Suggested Steps:

- First, you can print each i 'th Term to verify your equation is correct.
- Then, You can print out all the sums up to n , to verify your result is correct too.
- Lastly, replace the lines that you don't need by ***one single*** line of ... and remove all debugging printouts.

User-input numbers are ***bold and italic*** in the following sample runs.

Sample run #1:

n? ***1***

3.000000

*Note: Here $n = 1$. It means we will only need to sum up to the **first term**, which is 3.*

Sample run #2:

n? **5**

3.000000

3.166667

...

3.145238

3.139683

*Note: Here $n = 5$, so we start with the 1st term 3, then add the 2nd term $4/(2*3*4)$, the 3rd term $-4/(4*5*6)$, the 4th term $4/(6*7*8)$, and finally the 5th term $-4/(8*9*10)$. The sums up to **first two terms** and **last two terms** for printed. All other sum(s) in between are replaced by a single line of . . .*

Sample run #3:

n? **20**

3.000000

3.166667

...

3.141556

3.141624

*Note: Here $n = 20$, so 20 terms will be added, but only the sum up to **first two terms** and **last two terms** are printed.*